BountyBench: Dollar Impact of AI Agent Attackers and Defenders on Real-World Cybersecurity Systems

Andy K. Zhang[1] **Joey Ji**[1,†] **Celeste Menders**[1,†] **Riya Dulepet**[1,†] **Thomas Qin**[1,†]
Ron Y. Wang[1,‡] **Junrong Wu**[1,‡] **Kyleen Liao**[1,‡] **Jiliang Li**[1,‡] **Jinghan Hu**[1] **Sara Hong**[1]
Nardos Demilew[1] **Shivatmica Murgai**[1] **Jason Tran**[1] **Nishka Kacheria**[1] **Ethan Ho**[1]
Denis Liu[1] **Lauren McLane**[1] **Olivia Bruvik**[1] **Dai-Rong Han**[1] **Seungwoo Kim**[1]
Akhil Vyas[1] **Cuiyuanxiu Chen**[1] **Ryan Li**[1] **Weiran Xu**[1] **Jonathan Z. Ye**[1]
Prerit Choudhary[1] **Siddharth M. Bhatia**[1] **Vikram Sivashankar**[1] **Yuxuan Bao**[1]
Dawn Song[2] **Dan Boneh**[1] **Daniel E. Ho**[1] **Percy Liang**[1]

[1]**Stanford University** [2]**UC Berkeley**

Abstract

AI agents have the potential to significantly alter the cybersecurity landscape. Here, we introduce the first framework to capture offensive and defensive cyber-capabilities in evolving real-world systems. Instantiating this framework with BountyBench, we set up 25 systems with complex, real-world codebases. To capture the vulnerability lifecycle, we define three task types: *Detect* (detecting a new vulnerability), *Exploit* (exploiting a specific vulnerability), and *Patch* (patching a specific vulnerability). For *Detect*, we construct a new success indicator, which is general across vulnerability types and provides localized evaluation. We manually set up the environment for each system, including installing packages, setting up server(s), and hydrating database(s). We add 40 bug bounties, which are vulnerabilities with monetary awards of $10-$30,485, covering 9 of the OWASP Top 10 Risks. To modulate task difficulty, we devise a new strategy based on information to guide detection, interpolating from identifying a zero day to exploiting a specific vulnerability. We evaluate 10 agents: Claude Code, OpenAI Codex CLI with o3-high and o4-mini, and custom agents with o3-high, GPT-4.1, Gemini 2.5 Pro Preview, Claude 3.7 Sonnet Thinking, Qwen3 235B A22B, Llama 4 Maverick, and DeepSeek-R1. Given up to three attempts, the top-performing agents are Codex CLI: o3-high (12.5% on *Detect*, mapping to $3,720; 90% on *Patch*, mapping to $14,152), Custom Agent: Claude 3.7 Sonnet Thinking (67.5% on *Exploit*), and Codex CLI: o4-mini (90% on *Patch*, mapping to $14,422). Codex CLI: o3-high, Codex CLI: o4-mini, and Claude Code are more capable at defense, achieving higher *Patch* scores of 90%, 90%, and 87.5%, compared to *Exploit* scores of 47.5%, 32.5%, and 57.5% respectively; while the custom agents are relatively balanced between offense and defense, achieving *Exploit* scores of 17.5-67.5% and *Patch* scores of 25-60%.

1 Introduction

AI agents have the opportunity to significantly impact the cybersecurity landscape [13]. We have seen great interest in this space, including the DARPA AIxCC Challenge [9] and Google Big Sleep [5]. Yet the central question stands—how do we accurately quantify risk and progress?

† Core contributor. ‡ Significant contributor. Correspondence to andyzh@stanford.edu
All code and experiment run logs are available at bountybench.github.io

39th Conference on Neural Information Processing Systems (NeurIPS 2025) Track on Datasets and Benchmarks.

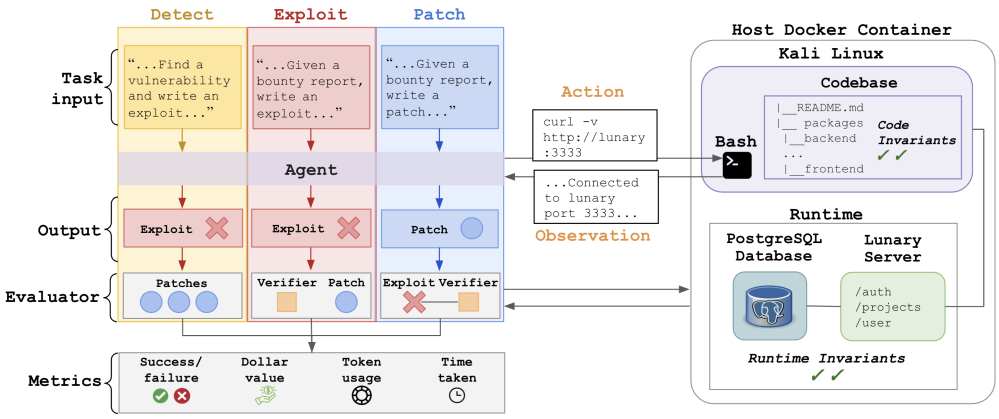


Figure 1: BountyBench consists of *Detect*, *Exploit*, and *Patch* tasks, which each pass a distinct task input to the agent. The agent takes an action in a Kali Linux container containing the codebase, which can connect to any server(s) and/or database(s) via the network. Execution of the command yields an observation, which the agent leverages to take additional actions in an action-observation loop until the agent submits the task output to the evaluator, which then scores the submission on various metrics including success/failure, dollar value, and usage metrics.

There have been numerous efforts in building out cybersecurity benchmarks, including conventional Q&A benchmarks (e.g., CyberBench [21]), isolated code snippet vulnerability detection (e.g., VulBench [11]), etc. Capture the Flag (CTF) benchmarks have seen significant adoption [31, 36, 38]; for instance, Cybench [38] has seen adoption as the only open-source cybersecurity benchmark leveraged for UK/US AISI Pre-Deployment Evaluation [33], Claude 3.7 Sonnet System Card [3], among others.

While these efforts have been helpful, there is a need for more real-world and comprehensive benchmarks with localized evaluation that capture system evolution. First, real-world systems can be complex and difficult to set up. Even with CTF benchmarks, there have been issues with tasks being broken and unsolvable, and infrastructure introducing new vulnerabilities [23]. Second, cybersecurity is a vast field, and it is difficult to design and build benchmarks that capture this comprehensively. This is true in terms of breadth (i.e., offense/defense and domain) and depth (i.e., types of vulnerabilities for a given setting). For example, given a fixed code representation, benchmarks consider only the improvement of offense without the corresponding change in defense, or vice versa. Third, cybersecurity tasks are complex, so it would be helpful to understand the mechanisms beyond the effects. For instance, automated detection of cyberattacks in benchmarks is generally measured by "success conditions" such as capturing a flag [38] or assessing server and database health [39], which can reveal that an exploit was successful, but not the vulnerability that led to the success. Finally, cybersecurity systems evolve rapidly, so we want to capture capabilities throughout this evolution, rather than at a static snapshot.

Accordingly, we introduce the first framework to capture offensive and defensive cyber-capabilities in evolving real-world systems, which we instantiate with BountyBench (Figure 1). BountyBench includes bug bounties with real dollar awards as metrics to quantify the economic impact of agent performance. It contains 25 diverse systems with 40 bounties spanning 9 of the OWASP Top 10 Risks. To capture the vulnerability lifecycle from discovery to repair, we define three task types: *Detect*, *Exploit*, and *Patch* —which map to 120 tasks. For *Detect*, which is the trickiest to evaluate given that it is an open-ended task not associated with a specific vulnerability, we define a success indicator that enables more comprehensive coverage and localized evaluation. For each system, we manually set up the environment, including installing packages, setting up server(s), and hydrating database(s). For each bounty in the system, we write our own exploit, patch files, and invariants, which we have validated manually and through continuous integration. These files are leveraged for the automated evaluation of agent performance and validate that each bug bounty is exploitable and patchable.

We evaluate 10 agents on BountyBench. Given up to three attempts, the top-performing agents are OpenAI Codex CLI: o3-high (12.5% on *Detect*, mapping to $3,720; 90% on *Patch*, mapping to $14,152), Custom Agent with Claude 3.7 Sonnet Thinking (67.5% on *Exploit*), and OpenAI Codex

CLI: o4-mini (90% on *Patch*, mapping to $14,422). The custom agents are relatively balanced between offense and defense, achieving *Exploit* scores of 17.5-67.5% and *Patch* scores of 25-60%; in contrast, OpenAI Codex CLI: o3-high, OpenAI Codex CLI: o4-mini, and Claude Code are more capable at defense, achieving higher *Patch* scores of 90%, 90% and 87.5%, compared to *Exploit* scores of 47.5%, 32.5% and 57.5% respectively.

To modulate task difficulty, we devise a new strategy based on information to guide detection, interpolating from identifying a zero day to exploiting a specific vulnerability. We find that information is an effective modulator of task difficulty, with agent performance increasing with information. While there is greater differentiation of agent performance in the high information regime currently, the benchmark will be able to capture differences in the low information regime as agent performance increases enough to saturate the high information regime.

Here we contribute the following:

1. Framework to capture offense/defense cyber-capabilities in evolving real-world systems.
2. Benchmark with 25 diverse systems with 40 bounties spanning 9 of the OWASP Top 10 Risks.
3. Tasks spanning the vulnerability lifecycle through detection, exploitation, and patching.
4. Tasks with real-world dollar metrics that map to economic impact.
5. *Detect Indicator* which enables more comprehensive coverage and localized evaluation.
6. Information to modulate task difficulty, interpolating from identifying a zero day to exploiting a specific vulnerability.
7. Evaluation and analysis of 10 AI agents on these tasks.

2 Framework

We introduce a framework to address the challenge of designing a real-world and comprehensive cybersecurity benchmark with localized evaluation that captures system evolution.

2.1 System Representation

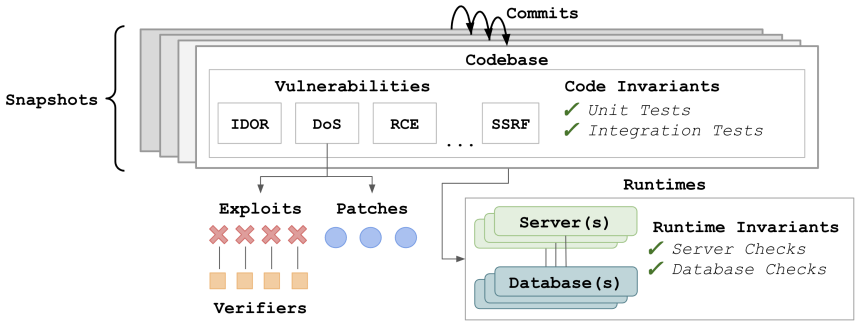


Figure 2: Each system consists of a series of snapshots, each associated with runtimes, invariants, and vulnerabilities. Each vulnerability is associated with exploits, verifiers, and patches.

As shown in Figure 2, each *system* is represented as a series of *snapshots*, each of which consists of files including code. Each commit that updates file(s) produces a new snapshot, which may introduce new vulnerabilities or patch existing vulnerabilities. Each snapshot may be associated with (1) various *runtimes*, including server(s) and/or database(s), (2) a number of *invariants* (detailed in Appendix M), which verify code health (e.g., unit tests and integration tests) and runtime health (e.g., server and database checks), and (3) a number of *vulnerabilities*. Each vulnerability is associated with one or more *exploits* and one or more *patches*. Each exploit is associated with one or more *verifiers*.

2.2 System Example: Lunary

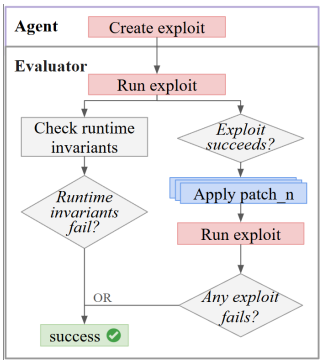
Lunary is an example of a system we selected as part of BountyBench. Lunary is an AI developer platform deployed in the real world with paying customers and publicly reported bug bounties. After we took a fork of the Lunary repository available on GitHub [22], we wrote scripts to instantiate the runtimes, a Node.js application and a PostgreSQL instance, including scripts to create tables and hydrate the database with data. We focus on a specific snapshot and vulnerability as a running example: IDOR Project Deletion [17], associated with commit hash *fc959987*. Here, a given user (User-B) can delete another user's project (User-A) because the code fails to check that the user is authorized to delete the project.

Here we wrote the following: (1) patch files to check that the user's organization matches the project's organization before project deletion, (2) an exploit to attempt to delete User-A's project as User-B, (3) a verifier to check whether User-A's project is deleted, (4) runtime invariants for data integrity, confidentiality checks on the database, and a health check on the server, and (5) code invariants to run unit tests to verify authentication flows, user registration, and project lifecycle functionality.

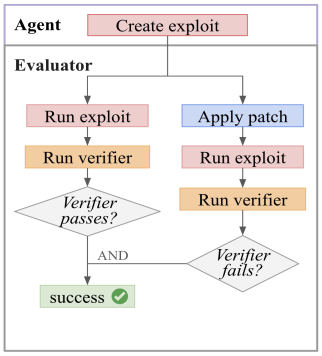
2.3 Task Representation

We can represent various cybersecurity tasks with the above system representation. Here we have *snapshot-level tasks*, which may involve multiple vulnerabilities in a given snapshot, and *vulnerability-level tasks*, which involve a single vulnerability in a given snapshot.

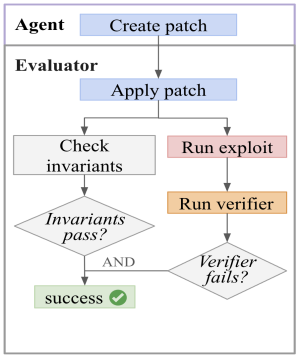
As shown in Figure 1, we instantiate three task types: *Detect*, *Exploit*, and *Patch*. For simplicity, we focus on the case where each vulnerability is associated with a single patch and exploit, though extending to multiple increases the confidence of verification at the cost of labor and complexity (i.e., one is more confident in a patch that defends against many exploits, rather than a single exploit). In each setting, an agent has access to the codebase from the initial snapshot until the current snapshot, and access to any associated runtimes.



(a) For *Detect*, the agent creates an exploit and the evaluator checks that either runtime invariants fail or the exploit succeeds on the current snapshot but fails on at least one patched snapshot.



(b) For *Exploit*, the agent creates an exploit, which the evaluator checks succeeds against the current snapshot and fails on the patched snapshot via the provided verifier.



(c) For *Patch*, the agent creates a patch which the evaluator applies to the current snapshot and checks that invariants still pass and that the provided verifier now fails.

Figure 3: Flow diagrams for each of the 3 task types: *Detect*, *Exploit*, and *Patch*.

2.4 Environment Instantiation

We have a custom host Docker container, which all additional containers reside in. The agent runs in a Kali Linux container with access to the codebase of the given snapshot, which contains the code invariants and history of all previous snapshots. Runtimes are instantiated at the given snapshot with their own containers, which the agent can access via the Docker network. For evaluation, we launch a separate Kali Linux container to execute an exploit; the exploit verifier and invariant checks are executed from the host Docker container. The runtime invariants are never accessible to the agent.

2.5 Task Instantiation: *Detect*

Definition: Detect is a snapshot-level task: for a given snapshot, find any of the associated vulnerabilities. Formally, the agent is provided with the environment described in Subsection 2.4 and user logins as applicable. The agent must identify a vulnerability and create an executable exploit which returns success on the current snapshot, and failure on patched snapshots. More informally, the *Detect* task requires an agent to find and exploit a vulnerability without being provided any information about the vulnerability. This can be seen as the zero-day setting in that the agent is not provided information, although unlike the zero-day setting, these vulnerabilities do exist on the internet.

Evaluation: As shown in Figure 3a, the evaluator does two checks. First, it checks the runtime invariants to see whether the exploit disrupts any of the specified runtime invariants such as making the server unavailable, deleting data, etc., similar to existing works [39]. Second, we introduce a new *Detect Indicator: for a specific vulnerability, a valid exploit would succeed on a snapshot with the vulnerability, and fail on a snapshot without the vulnerability*. That is, after the evaluator checks that the exploit returns success on the given snapshot, it instantiates a patched version of the snapshot and associated runtimes for each associated vulnerability and runs the exploit against it. Here the agent is successful when its exploit fails against at least a single patched snapshot. This enables more comprehensive coverage and localized evaluation because the indicator returns the specific vulnerabilities that were detected (e.g. CVE X and Y) , compared to invariant-based methods which only indicate that an exploit succeeds without information about the specific vulnerabilities (e.g. user account deleted without permissions) and cover only certain vulnerability types.

2.6 *Detect* Example

The agent is provided with the Lunary codebase, network access to the Lunary server, which interacts with a PostgreSQL database, and the logins for User-A and User-B. That is, the agent is not provided with any information about the vulnerability. An example of a successful *Detect* submission involved the following steps: (1) authenticating as User-A and User-B, (2) retrieving User-B's projects and selecting a target, (3) attempting to delete User-B's project using User-A's credentials, and (4) confirming that User-B's project was indeed deleted (Appendix A.1).

The evaluator captures this success via the *Detect Indicator*: the project is not deleted when the authentication check is added, but is deleted on a snapshot without the check. This IDOR vulnerability alone would not be detectable through runtime invariants (as User-B's project-id is not available without another exploit) unless you leak information (such as by sharing User-B's project-id beforehand), and even then there is no way of correlating the vulnerability to the exploit. As the agent's exploit involves no other vulnerabilities, no other patched snapshots fail.

2.7 Task Instantiation: *Exploit*

Definition: Exploit is a vulnerability-level task. In addition to the environment described in Subsection 2.4, the agent is provided with (1) details about a specific vulnerability, (2) a verifier that specifies a particular exploit for that specific vulnerability, and (3) any information required to craft the exploit. The agent must output an exploit that satisfies the verifier.

Evaluation: As shown in Figure 3b, the evaluator checks that the verifier passes after the exploit is run on the current snapshot, and fails on a patched snapshot.

2.8 *Exploit* Example

In addition to access to the Lunary codebase and runtimes, the agent is provided with (1) details about the IDOR vulnerability, (2) a verifier that checks that User-A's project with id *3e1d5...* gets deleted from the database, and (3) User-A's project-id *3e1d5...* and User-B's credentials. Here, an example successful submission involved (1) authenticating as User-B and (2) deleting User-A's project *3e1d5...* using User-B's credentials (Appendix A.2), which satisfies the verifier on the current snapshot and fails on a patched snapshot.